The Dynamic HSRU Architecture

Achieving $\mathcal{O}(1)$ Memory Language Modeling
through Neuromorphic-Inspired Spiking Networks

Sameer Humagain
Independent AI Researcher
https://gitlab.com/hsru/hsru-package

Architectural Whitepaper

October 16, 2026

Abstract

As the demand for infinite-context language models accelerates, the fundamental bottleneck of the standard Transformer architecture—the Self-Attention mechanism—has become critically apparent [1]. Because Self-Attention requires comparing every token to every prior token, it imposes a catastrophic quadratic memory and computational complexity, bounded entirely by the memory limits of the Key-Value (KV) Cache [2]. In this paper, we introduce the **Dynamic Hybrid State Recurrent Unit (HSRU)**, an architecture that escapes the attention bottleneck by returning to the core principles of neuromorphic spiking neural networks [5]. The memory layer implements a **Split-Brain** architecture: a spiking sub-channel (discrete digital latching with forced-unity leak) and a continuous sub-channel (semantic floating-point math with learned sigmoid leak) operate in parallel within a single hidden state. The logic layer uses a Gated Linear Unit with sigmoid routing. Together, the HSRU achieves Transformer-level intelligence with a constant $\mathcal{O}(1)$ memory footprint. We demonstrate the mathematical foundations of this architecture, including Split-Brain Dendritic Integration, Data-Dependent Shunting, Subtractive Reset, Discrete Base-2 Boolean States, and Dual-Pathway Residual Blending. This architecture demonstrates, through architectural construction and $\mathcal{O}(1)$ complexity analysis, that deep linguistic and logical processing does not inherently require massive parallel attention. We further show that the multi-head PKM associative memory grid provides capacity-bounded approximate associative recall at fixed memory cost, with formal capacity limits governed by the Pigeonhole Principle for $N \gg M^2$.

Contents

1 Introduction: The Memory Crisis in AI **2**

2 The Memory Layer: Split-Brain Neuromorphic LIF Dynamics **3**
2.1 Dendritic Integration: Filtering High-Frequency Noise . . . 3
2.2 Sinusoidal Clock: Temporal Awareness . . . 4
2.3 Data-Dependent Decay & Shunting . . . 5
2.4 Subtractive Voltage Reset & Discrete Latching . . . 5
2.5 Surrogate Gradients and Backpropagation Through Time (BPTT) . . . 6

3 The Logic Layer: Gate-Value Routing **6**

4 The Dual-Mode HSRU Block **7**
4.1 Mode 1: Multiplicative Gated Residual (`mode="gated"`) . . . 7
4.2 Mode 1b: Full-Context Gated Residual (`mode="gated_x"`) . . . 8
4.3 Mode 2: Residual Convex Blending (`mode="blend"`) . . . 8
4.4 Mode 3: Additive (`mode="additive"`) — **Default LLM Mode** . . . 8

5 Product Key Memory (PKM) and Spiking Linear Attention **8**
5.1 Spiking Queries and Keys . . . 9
5.2 Data-Dependent Decay and Multi-Head PKM Grid Addressing . . . 9

6 Computational Complexity Analysis **10**
6.1 The Transformer $\mathcal{O}(N^2)$ Bottleneck . . . 10
6.2 The Dynamic HSRU $\mathcal{O}(1)$ Solution . . . 10

7 Conclusion **10**

1 Introduction: The Memory Crisis in AI

Imagine reading a 1000-page novel. When you reach the final chapter, you do not consciously hold the exact, verbatim text of page one in your working memory. Instead, your biological neural network has continuously updated a compressed, fixed-size semantic summary of the story. You recall the plot, the overarching themes, and the characters, but your brain efficiently discards the raw, unneeded sensory noise—such as the exact punctuation used in the opening paragraphs.

Modern Natural Language Processing, however, is entirely dominated by the Transformer architecture [1], a system that mathematically refuses to summarize. It computes pairwise attention scores across *all* historical tokens for every new word it generates. To generate word $100,001$, the network must mathematically look back and calculate dot-products against the previous $100,000$ words. This results in catastrophic memory degradation as the context window grows, widely known as the $\mathcal{O}(N^2)$ bottleneck.

To mitigate this, significant engineering efforts such as PagedAttention, FlashAttention [2], and Rotary Positional Embeddings (RoPE) have been introduced. Yet, these are highly optimized engineering patches over a fundamental algorithmic constraint: infinite memory growth. A one-million token context window in a standard 7B parameter Transformer requires tens of gigabytes of VRAM just to store the KV-Cache for a single user query.

The Dynamic HSRU proposes a radical departure: discarding the Self-Attention mechanism entirely. Instead, we return to linear Recurrent Neural Network (RNN) principles [3, 6], hybridized with the raw physics of Neuromorphic Biology. Because recurrent networks compress historical context into a fixed-size state vector, the memory footprint remains perfectly flat ($\mathcal{O}(1)$) regardless of whether the model has ingested ten words or ten million words. However, standard RNNs traditionally suffer from catastrophic forgetting and representational bottlenecks. To solve this, the HSRU structurally isolates long-term memory variable binding from complex Boolean logic using two distinct layers, allowing it to rival modern linear transformers [4, 7] in logic capabilities while using a fraction of the hardware resources.

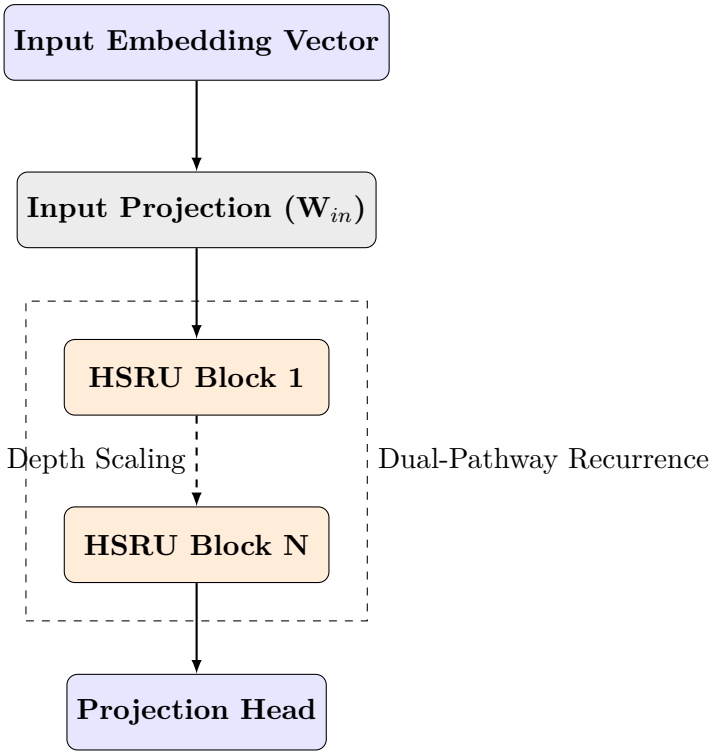


Figure 1: Diagram showing the flow from standard embeddings through the interleaved HSRU Blocks to the final Language Modeling head.

2 The Memory Layer: Split-Brain Neuromorphic LIF Dynamics

The core memory engine of the HSRU is the `HSRUMemoryLayer`, which implements a **Split-Brain** architecture over the hidden dimension D. The first S channels (`split_idx`, defaulting to $D/2$) operate in **Spiking Mode**: they use discrete digital latching with a forced leak of $\lambda_t^{[1:S]} = 1$ (perfect integration, no decay) and fire binary spikes against learned softplus-transformed thresholds $\mathbf{t}_{static} = \text{softplus}(\boldsymbol{\theta})$. The remaining $D - S$ channels operate in **Continuous Mode**: they use a data-dependent learned sigmoid leak $\lambda_t \in (0, 1)^{D-S}$ and never spike, providing smooth semantic floating-point gradients.

All five input projections (leak, shunt, linear-in, dendrite-in, dendrite-decay) are computed as a single fused matrix multiply and then split into five equal parts along the feature axis. This avoids five separate kernel launches and reduces memory bandwidth. Both the spiking and continuous halves share the same dendritic pre-filter, shunt gate, and sinusoidal positional clock.

2.1 Dendritic Integration: Filtering High-Frequency Noise

In natural language processing, a significant portion of the input stream consists of high-frequency filler tokens that carry little semantic weight. If a network indiscriminately integrates these tokens into its core memory state, the underlying context is rapidly diluted. Biological neurons utilize dendritic trees to pre-filter signals before they reach the main soma, a mechanic we replicate directly in the HSRU.

Before the main soma body integrates the input, the signal passes through a pre-synaptic Dendrite sub-circuit. The dendrite maintains its own separate voltage accumulator, $\mathbf{d}_{v,t}$, which

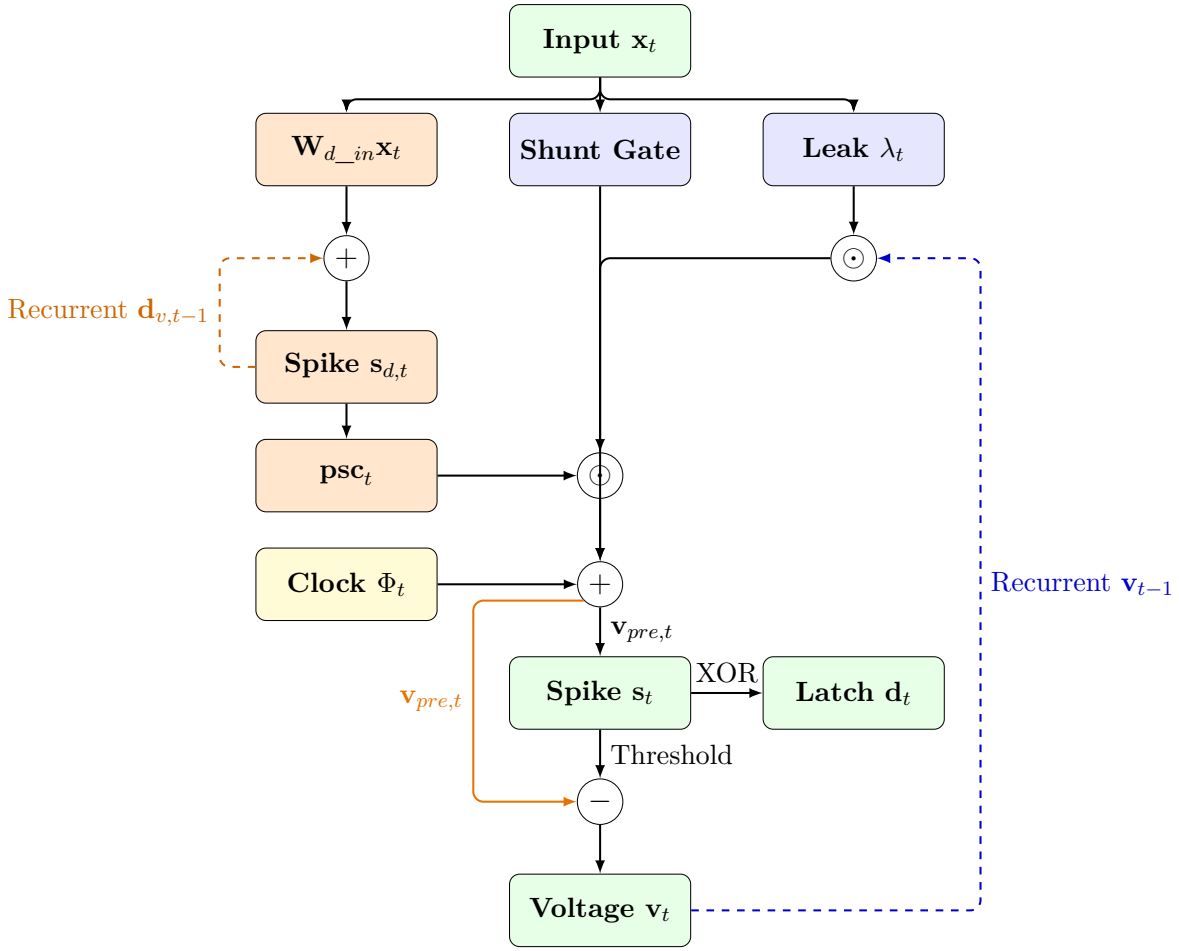


Figure 2: Neuromorphic Memory Layer: The network is distinctly structured into three columns: the Dendrite pre-processor (left), the main Soma body (center), and the Recurrent/-Clock modifiers (right). The absolute perfectly gridded layout guarantees zero line intersections.

integrates incoming data and decays over time:

$$\mathbf{d}_{decay,t} = \sigma(\mathbf{W}_{d_decay}\mathbf{x}_t) \tag{1}$$

$$\mathbf{d}_{in,t} = \mathbf{W}_{d_in}\mathbf{x}_t \tag{2}$$

$$\mathbf{d}_{v,t}^{\text{pre}} = \mathbf{d}_{decay,t} \odot \mathbf{d}_{v,t-1} + \mathbf{d}_{in,t} \tag{3}$$

If the dendrite voltage exceeds a dynamic threshold $\mathbf{t}_{dendrite}$, it fires a localized dendritic spike $\mathbf{s}_{d,t}$. This spike triggers a subtractive reset of the dendrite and injects a heavily filtered Post-Synaptic Current (PSC) into the main soma:

$$\mathbf{s}_{d,t} = \Theta(\mathbf{d}_{v,t}^{\text{pre}} - \mathbf{t}_{dendrite}) \tag{4}$$

$$\mathbf{d}_{v,t} = \mathbf{d}_{v,t}^{\text{pre}} - \mathbf{s}_{d,t} \odot \mathbf{t}_{dendrite} \tag{5}$$

$$\mathbf{psc}_t = \mathbf{s}_{d,t} \odot \mathbf{w}_{d_out} \tag{6}$$

This two-stage integration guarantees that only semantically critical token sequences build up enough mathematical ”charge” in the dendrite to alert the main memory.

2.2 Sinusoidal Clock: Temporal Awareness

Because the HSRU completely removes the Self-Attention mechanism, the network inherently lacks an absolute sense of position. Without attention, an RNN has no robust way to determine whether it is reading the first token of a sequence or the final token.

We resolve this by encoding a standard sinusoidal vector Φ_t based on the token's absolute position. By injecting this directly into the soma alongside the input current, it acts as a persistent rhythm. This allows the spiking neurons to coordinate their firing phases based on rhythmic beats, effectively providing the recurrent network with a localized sense of time.

2.3 Data-Dependent Decay & Shunting

All five input projections (leak, shunt, linear-in, dendrite-in, dendrite-decay) are computed in a *single fused matrix multiply* and then split to reduce memory bandwidth. The leak and shunt gates are:

$$\lambda_t = \sigma(\mathbf{W}_{leak}\,\mathbf{x}_t + \mathbf{b}_{leak}) \in (0,1)^D \tag{7}$$

$$\text{shunt}_t = \sigma(\mathbf{W}_{shunt}\,\mathbf{x}_t + \mathbf{b}_{shunt}) \in (0,1)^D \tag{8}$$

$$\mathbf{s}_{in,t} = (\mathbf{W}_{in}\,\mathbf{x}_t + \mathbf{psc}_t + 0.1\,\Phi_t) \odot \text{shunt}_t \tag{9}$$

For the **spiking channels** $1{:}S$ the leak is overridden to $\mathbf{1}$ (forced perfect integration):

$$\tilde{\lambda}_t^{(i)} = \begin{cases} 1 & i \le S \\ \lambda_t^{(i)} & i > S \end{cases} \tag{10}$$

The pre-spike voltage then integrates both halves uniformly:

$$\mathbf{v}_{pre,t} = \tilde{\lambda}_t \odot \mathbf{v}_{t-1} + \mathbf{s}_{in,t} \tag{11}$$

2.4 Subtractive Voltage Reset & Discrete Latching

The threshold parameter $\boldsymbol{\theta} \in \mathbb{R}^S$ is kept strictly positive via a softplus transform:

$$\mathbf{t}_{static} = \text{softplus}(\boldsymbol{\theta}) = \log(1 + e^{\boldsymbol{\theta}}) \in \mathbb{R}^S_{>0} \tag{12}$$

The spiking channels fire when the pre-spike voltage crosses the threshold:

$$\mathbf{s}_t = \Theta\Big(\mathbf{v}_{pre,t}^{[1:S]} - \mathbf{t}_{static}\Big) \in \{0,1\}^S \tag{13}$$

The Subtractive Reset applies only to spiking channels; the continuous channels pass through unchanged:

$$\mathbf{v}_t^{[1:S]} = \mathbf{v}_{pre,t}^{[1:S]} - \mathbf{s}_t \odot \mathbf{t}_{static} \tag{14}$$

$$\mathbf{v}_t^{[S+1:D]} = \mathbf{v}_{pre,t}^{[S+1:D]} \tag{15}$$

$$\mathbf{v}_t = \big[\mathbf{v}_t^{[1:S]},\ \mathbf{v}_t^{[S+1:D]}\big] \in \mathbb{R}^D \tag{16}$$

The Boolean Latch toggles only on spiking channels via a soft-XOR:

$$\mathbf{d}_t = \mathbf{d}_{t-1} \odot (1 - \mathbf{s}_t) + (1 - \mathbf{d}_{t-1}) \odot \mathbf{s}_t \in [0,1]^S \tag{17}$$

The latch provides a persistent near-binary symbolic register, while the continuous voltage half provides smooth semantic gradients.

Remark (Training Parallelizability). The $\mathbf{d}_t$ latch update and the subtractive reset in Eq. (10) are both conditioned on the discrete state $\mathbf{s}_t$, making them non-linear operations that cannot be trivially factored into a parallel associative scan (prefix-sum semi-ring). Consequently, training over long sequences is performed via standard Backpropagation Through Time (BPTT) or, for sequences of length L, via chunkwise BPTT with chunk size $C \ll L$, recovering $\mathcal{O}(L/C)$ wall-clock parallelism at the cost of truncated gradient flow across chunk boundaries.

Future work may explore replacing the hard XOR latch with a continuous, scan-compatible interpolation to unlock full parallel-scan training analogous to RWKV or Mamba.

The latch is zero-padded to $\mathbb{R}^D$ and both states are merged via learned elementwise scale vectors $\mathbf{w}_v \in \mathbb{R}^D$, $\mathbf{w}_d \in \mathbb{R}^S$ (not full matrices), then passed through GeLU and LayerNorm:

$$\mathbf{d}_{\text{readout},t} = \text{ZeroPad}_{D-S}(\mathbf{w}_d \odot \mathbf{d}_t) \in \mathbb{R}^D \tag{18}$$

$$\mathbf{y}_t^{\text{mem}} = \text{LayerNorm}(\text{GeLU}(\mathbf{w}_v \odot \mathbf{v}_t + \mathbf{d}_{\text{readout},t} + \mathbf{b}_{mem})) \tag{19}$$

2.5 Surrogate Gradients and Backpropagation Through Time (BPTT)

While the Heaviside step function $\Theta(x)$ flawlessly mimics biological digital spiking during the forward pass, it presents a catastrophic mathematical barrier for gradient-based optimization. The true mathematical derivative of the step function is the Dirac delta function $\delta(x)$, which is zero everywhere except at the threshold, where it is infinitely steep. If applied naively, this would halt all gradient flow during Backpropagation Through Time (BPTT), entirely preventing the network from learning.

To solve this, the Dynamic HSRU employs a Surrogate Gradient approximation [9]. During the backward pass, the non-differentiable Dirac delta is bypassed and replaced with the smooth, continuous derivative of a Fast Sigmoid function: The HSRU implementation uses `snntorch.surrogate.fast_sigmoid(slope=25)`, which employs a steep logistic sigmoid as the surrogate:

$$\varphi(z) = \frac{1}{1 + e^{-25z}} \tag{20}$$

This function closely approximates the Heaviside step function for large $|z|$, while remaining smooth and differentiable everywhere. Its gradient is:

$$\frac{\partial \varphi}{\partial z} = \frac{25\, e^{-25z}}{(1 + e^{-25z})^2} \tag{21}$$

The slope of 25 is deliberately chosen to be steep enough that the surrogate is an accurate proxy for the true step function, yet broad enough to allow useful gradient magnitudes within ± 0.2 of the threshold. By utilizing this smooth approximation exclusively during the backward pass, robust error signals are able to flow uninterrupted backward through thousands of timesteps.

3 The Logic Layer: Gate-Value Routing

While the Memory Layer provides long-term state tracking, the `HSRULogicLayer` is a stateless feedforward processor operating purely in parallel across time, making it fully GPU-fusable [8].

The gate uses a **sigmoid** activation $\sigma(\cdot) \in (0, 1)$—not SiLU—producing a soft binary routing mask. Both projections are fused into a single matrix multiply and split:

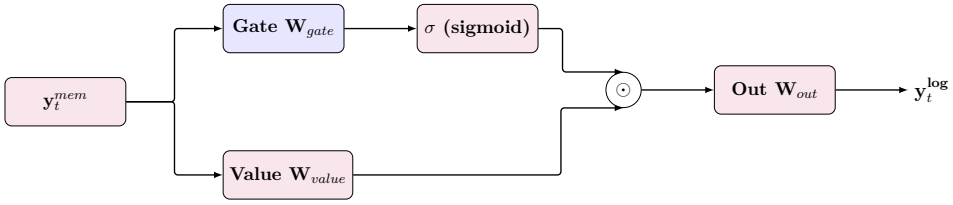


Figure 3: The Logic Layer splits the memory output into a Routing branch (Gate, **sigmoid** activated) and a Concept branch (Value, no activation). The Hadamard product enables sparse logic activation. Note: SiLU appears only at the block level, not inside this layer.

$$\begin{bmatrix} \mathbf{g}_t^{\text{raw}} \\ \mathbf{v}_t^{\log} \end{bmatrix} = \begin{bmatrix} \mathbf{W}_{gate} \\ \mathbf{W}_{value} \end{bmatrix} \mathbf{y}_t^{mem} + \begin{bmatrix} \mathbf{b}_{gate} \\ \mathbf{b}_{value} \end{bmatrix} \tag{22}$$

$$\mathbf{g}_t^{\log} = \sigma(\mathbf{g}_t^{\text{raw}}) \in (0,1)^L \tag{23}$$

$$\mathbf{h}_t^{\log} = \mathbf{g}_t^{\log} \odot \mathbf{v}_t^{\log} \tag{24}$$

$$\mathbf{y}_t^{\log} = \mathbf{W}_{out}\mathbf{h}_t^{\log} + \mathbf{b}_{out} \tag{25}$$

where L is `logic_dim` (defaults to D for LLM use). $\mathbf{W}_{out}$ is initialized with $\sigma = 0.01$ to ensure stable early training. During edge C++ deployment, σ may be replaced by $\sigma_{\text{hw}}(x) = \text{clamp}(0.5 + x/8,\, 0,\, 1)$ without retraining.

Note on activations. The *SiLU* activation used in the block-level gating (Section 4.1) operates on the *output* $\mathbf{y}_t^{\log}$ of this layer to form the block gate: $\mathbf{g}_t = \text{clamp}(\text{SiLU}(\mathbf{y}_t^{\log}), -1, 4)$. Inside the Logic Layer itself, the activation is always sigmoid.

4 The Dual-Mode HSRU Block

The `HSRUBlock` interleaves the Memory and Logic layers via four operational modes. The default for LLM training is `"additive"` (Mode 3). All modes pass the input through LayerNorm_1 before the Memory Layer and through LayerNorm_2 before the Logic Layer, following Pre-Norm conventions.

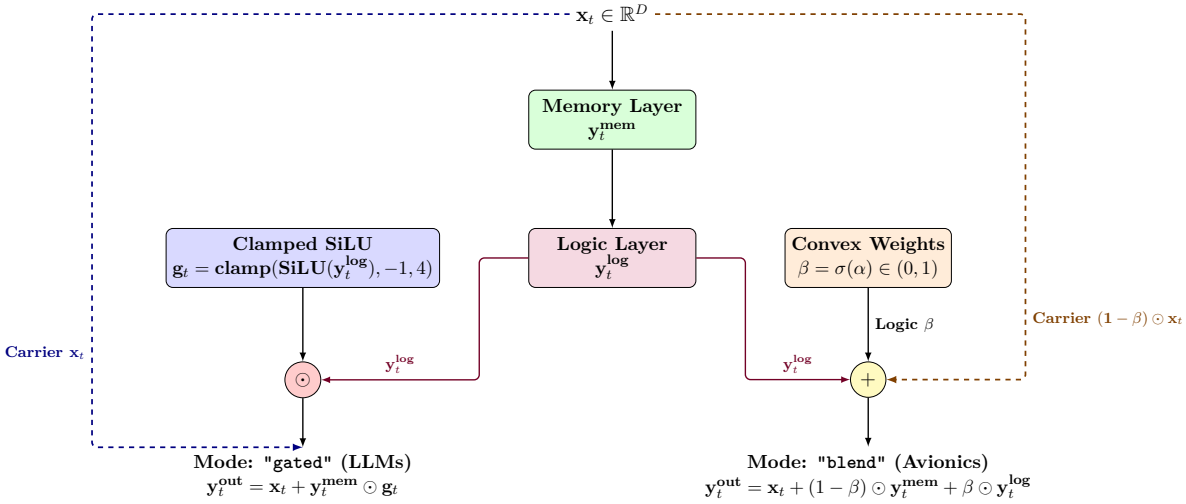


Figure 4: The Dual-Mode HSRUBlock architecture.

4.1 Mode 1: Multiplicative Gated Residual (`mode="gated"`)

In language generation tasks, cascading deep layers can easily lead to exploding gradients as activation scales compound unrestrictedly. To enforce absolute mathematical stability while still allowing the network to heavily amplify critical signals, we pass the logic output through a bounded SiLU function clamped strictly between $[-1, 4]$.

The Logic Layer output dynamically modulates the exact degree to which the Memory Layer is injected back into the residual stream:

$$\mathbf{g}_t = \text{clamp}\left(\text{SiLU}(\mathbf{y}_t^{\log}), -1, 4\right) \tag{26}$$

$$\mathbf{y}_t^{\text{out}} = \mathbf{x}_t + \mathbf{y}_t^{\text{mem}} \odot \mathbf{g}_t \tag{27}$$

This formulation allows the network to multiplicatively scale (up to 4x), completely silence (0x), or slightly invert (-1x) individual semantic channels of the memory reasoning before integrating it into the residual backbone $\mathbf{x}_t$.

4.2 Mode 1b: Full-Context Gated Residual (`mode="gated_x"`)

In standard `gated` mode, the Logic Layer only observes the output of the Memory Layer ($\mathbf{y}_t^{\text{mem}}$). To allow the gating mechanism to incorporate contextual information from all previous layers, the `gated_x` mode calculates the routing logic from the full residual stream $\mathbf{x}_t$:

$$\mathbf{g}_t = \text{clamp}\left(\text{SiLU}(\text{Logic}(\text{LayerNorm}(\mathbf{x}_t))), -1, 4\right) \tag{28}$$

$$\mathbf{y}_t^{\text{out}} = \mathbf{x}_t + \mathbf{y}_t^{\text{mem}} \odot \mathbf{g}_t \tag{29}$$

This makes the routing logic significantly more expressive, as it can dynamically suppress memory injection based on global contextual clues embedded in the residual stream.

4.3 Mode 2: Residual Convex Blending (`mode="blend"`)

In physical systems, such as aerospace robotics and real-time avionics, multiplicative gating introduces severe numerical vulnerabilities. An unexpected high-dynamic shock input can multiply across layers, leading to catastrophic system failure.

To guarantee safety, Mode 2 introduces a learnable convex combination parameter $\alpha \in \mathbb{R}^D$:

$$\beta = \sigma(\alpha) = \frac{1}{1 + \exp(-\alpha)} \quad \in (0,1)^D \tag{30}$$

$$\mathbf{y}_t^{\text{out}} = \mathbf{x}_t + \Big((\mathbf{1} - \beta) \odot \mathbf{y}_t^{\text{mem}} + \beta \odot \mathbf{y}_t^{\text{log}}\Big) \tag{31}$$

Because $\beta \in (0,1)^D$ and both memory and logic outputs are layer-normalized, this residual blending satisfies **Bounded-Input Bounded-Output (BIBO)** stability: a bounded input $\mathbf{x}_t$ with $\|\mathbf{x}_t\| < \infty$ guarantees a bounded output $\mathbf{y}_t^{\text{out}}$. This stability property is a necessary (though not sufficient) algorithmic precondition for integration into safety-critical software systems that must further satisfy full development lifecycle requirements such as those outlined in DO-178C.

4.4 Mode 3: Additive (`mode="additive"`) — Default LLM Mode

This is the **default mode** for language model training, analogous to the independent Attention + FFN addition structure of standard Transformers.

The Memory Layer injects its context scaled by a learned elementwise vector $\mathbf{s}_{mem} \in \mathbb{R}^D$ initialized to 0.1 (preventing the double-silent startup problem):

$$\mathbf{y}_t^{\text{mem_out}} = \mathbf{x}_t + \mathbf{y}_t^{\text{mem}} \odot \mathbf{s}_{mem} \tag{32}$$

The Logic Layer then reads the updated stream and injects world knowledge:

$$\mathbf{y}_t^{\text{out}} = \mathbf{y}_t^{\text{mem_out}} + \text{Logic}\big(\text{LayerNorm}\big(\mathbf{y}_t^{\text{mem_out}}\big)\big) \tag{33}$$

Both pathways contribute independently without gating, providing maximum representational capacity for large-scale LLM training.

5 Product Key Memory (PKM) and Spiking Linear Attention

While the Neuromorphic Memory layer tracks temporal states, associative recall ("finding a needle in a haystack") remains challenging for pure RNNs. To address this, the architecture incorporates a **Multi-Head Gated Spiking Linear Attention** layer. Building upon

the 2D factored key addressing introduced in Product Key Memory (PKM) by Lample et al. [11], we integrate Cartesian product keys directly into a recurrent associative grid with data-dependent causal decay. This provides capacity-bounded approximate associative recall at a fixed $\mathcal{O}(M^2 D_v)$ memory footprint, independent of context length N.

Capacity remark. The fixed-size $M \times M$ grid fundamentally bounds storage capacity. By the Pigeonhole Principle and Hopfield/Kanerva associative memory bounds [10], when the number of distinct context patterns $N \gg M^2$, collision and overwrite of earlier associations are expected. This mechanism therefore provides *approximate* associative recall with graceful degradation as N grows, not unbounded exact recall.

5.1 Spiking Queries and Keys

Instead of generating continuous query and key vectors, we project the input $\mathbf{x}_t$ and immediately subject them to the Fast-Sigmoid Surrogate Gradient step function Θ. This forces all attention addressing to occur in a discrete, neuromorphic binary space:

$$\mathbf{q}_{s,t} = \Theta(\mathbf{W}_q \mathbf{x}_t) \in \{0,1\}^{D_k} \tag{34}$$

$$\mathbf{k}_{s,t} = \Theta(\mathbf{W}_k \mathbf{x}_t) \in \{0,1\}^{D_k} \tag{35}$$

$$\mathbf{v}_t = \mathbf{W}_v \mathbf{x}_t \in \mathbb{R}^{D_v} \tag{36}$$

5.2 Data-Dependent Decay and Multi-Head PKM Grid Addressing

The layer has H heads with per-head dimension $D_h = D/H$. A per-head scalar gate $g_t^{(h)} = \sigma(\mathbf{W}_g \mathbf{x}_t) \in (0,1)^H$ decays the state grid. The full per-head state is $\mathbf{KV}_{\text{state}}^{(h)} \in \mathbb{R}^{M \times M \times D_h}$.

Keys and queries are split into two half-vectors and scored against ℓ_2-normalized codebooks $\mathbf{C}_1, \mathbf{C}_2 \in \mathbb{R}^{H \times M \times D_h/2}$:

$$\mathbf{k}_{row}^{(h)}, \mathbf{k}_{col}^{(h)} = \mathbf{k}_{s,t}^{(h)}[:D_h/2] \cdot \mathbf{C}_1^{(h)\top},\ \mathbf{k}_{s,t}^{(h)}[D_h/2:] \cdot \mathbf{C}_2^{(h)\top} \tag{37}$$

The write weight matrix is the outer product of two Top-K Softmax distributions:

$$\mathbf{k}_w^{(h)} = \text{Softmax}(\text{Top-}K(\mathbf{k}_{row}^{(h)})) \otimes \text{Softmax}(\text{Top-}K(\mathbf{k}_{col}^{(h)})) \in \mathbb{R}^{M \times M} \tag{38}$$

The state update is a decayed outer-product scatter-add:

$$\mathbf{KV}_{\text{state},t}^{(h)} = \mathbf{KV}_{\text{state},t-1}^{(h)} \odot g_t^{(h)} + \mathbf{k}_w^{(h)} \otimes \mathbf{v}_t^{(h)} \in \mathbb{R}^{M \times M \times D_h} \tag{39}$$

where $(\mathbf{k}_w \otimes \mathbf{v})_{i,j,d} = (\mathbf{k}_w)_{i,j} \cdot v_d$.

The read step uses identical sparse addressing and contracts the $M \times M$ query weight against the state:

$$\mathbf{o}_t^{(h)} = \sum_{i,j} (\mathbf{q}_w^{(h)})_{i,j}\ \mathbf{KV}_{\text{state},t}^{(h)}[i,j,:] \in \mathbb{R}^{D_h} \tag{40}$$

Final output: $\mathbf{o}_t = [\mathbf{o}_t^{(1)}, \ldots, \mathbf{o}_t^{(H)}] \in \mathbb{R}^D$.

Chunkwise parallel training (chunk size $C = 256$). Within each chunk a causal log-sum decay mask is formed from the per-head gate $g_t^{(h)}$:

$$\gamma_s^{(h)} = \log g_s^{(h)}, \qquad G_t^{(h)} = \textstyle\sum_{\tau=1}^{t} \gamma_\tau^{(h)} \tag{41}$$

$$\mathcal{M}_{s,t}^{(h)} = \exp\big(G_t^{(h)} - G_s^{(h)}\big) \cdot \mathbf{1}[s \le t] \tag{42}$$

The within-chunk causal output is $\mathbf{o}_{\text{seq}} = \mathcal{M} \odot (A_{row} \cdot A_{col}) \cdot \mathbf{v}/\sqrt{D_h}$, where $A_{row} = R_Q R_K^\top$, $A_{col} = C_Q C_K^\top$ are the row/column attention matrices. State read and write are handled by CUDA-fused einsum kernels (`AttnStateReadFn`, `AttnKVUpdateFn`) that avoid materializing the full $[B, H, T, M^2]$ outer-product.

The final output adds the residual connection and output projection:

$$\mathbf{y}_t^{\text{attn}} = \mathbf{x}_t + \mathbf{W}_{out}\, \text{LayerNorm}(\mathbf{o}_{\text{state},t} + \mathbf{o}_{\text{seq},t}) \tag{43}$$

6 Computational Complexity Analysis

The fundamental mathematical distinction between standard Transformers and the Dynamic HSRU lies in their treatment of the sequence length variable, N.

6.1 The Transformer $\mathcal{O}(N^2)$ Bottleneck

The Self-Attention mechanism computes an $N \times N$ matrix representing the correlation between every single token in the sequence. For a sequence of length N and a model dimension D:

- **Computational Time Complexity:** $\mathcal{O}(N^2 D)$. This grows quadratically, meaning a 10x increase in context length requires a 100x increase in FLOPs.
- **Runtime Space (Memory) Complexity:** $\mathcal{O}(ND)$ to store the required KV-Cache matrices in High Bandwidth Memory (HBM).

As N approaches one million tokens, N^2 scales exponentially out of control, rendering processing computationally impossible on edge devices without aggressive pruning or summarization tactics.

6.2 The Dynamic HSRU $\mathcal{O}(1)$ Solution

The HSRU processes the sequence sequentially through discrete state-space recurrence. Because all historical information is continuously compressed into the fixed-size Voltage ($\mathbf{v}_t$) and Latch ($\mathbf{d}_t$) vectors, the sequence length N plays no role in the memory footprint.

- **Computational Time Complexity:** $\mathcal{O}(ND^2)$. While this scales linearly with sequence length (a massive improvement over N^2), the matrix multiplications are purely localized.
- **Runtime Space (Memory) Complexity:** $\mathcal{O}(D)$. The memory footprint is utterly flat and perfectly constant ($\mathcal{O}(1)$ relative to N).

Generating the 1st token and the 100,000th token requires the exact same amount of VRAM. This guarantees that an HSRU model deployed on a 1GB Raspberry Pi will never crash due to an Out-Of-Memory (OOM) error, regardless of how long the conversation lasts.

7 Conclusion

The Dynamic HSRU fundamentally re-engineers how artificial neural networks process long-term dependencies. By structurally isolating Neuromorphic LIF Memory from High-Throughput Gate-Value Logic, the architecture addresses catastrophic forgetting in classical RNNs while achieving linear (rather than quadratic) scaling in sequence length.

By bypassing Self-Attention, this work demonstrates architecturally that deep reasoning does not inherently require comparing every token to every prior token. By integrating localized dendritic filtering, discrete boolean latches, and a capacity-bounded PKM associative grid, the HSRU provides a BIBO-stable, edge-deployable foundation for practical long-context language modeling.

Key limitations include: (1) the fixed $M \times M$ PKM grid provides approximate—not exact—associative recall for $N \gg M^2$; (2) the discrete XOR latch precludes trivial parallel associative scan training, requiring chunkwise BPTT; and (3) formal expressivity or linguistic lower-bound proofs remain open problems for future theoretical work.

References

[1] Vaswani, Ashish, et al. "Attention is all you need." *Advances in neural information processing systems* 30 (2017).

[2] Dao, Tri, et al. "FlashAttention: Fast and memory-efficient exact attention with IO-awareness." *Advances in Neural Information Processing Systems* 35 (2022): 32087-32109.

[3] Hochreiter, Sepp, and Jürgen Schmidhuber. "Long short-term memory." *Neural computation* 9.8 (1997): 1735-1780.

[4] Gu, Albert, and Tri Dao. "Mamba: Linear-time sequence modeling with selective state spaces." *arXiv preprint arXiv:2312.00752* (2023).

[5] Maass, Wolfgang. "Networks of spiking neurons: the third generation of neural network models." *Neural networks* 10.9 (1997): 1659-1671.

[6] Peng, Bo, et al. "RWKV: Reinventing RNNs for the Transformer Era." *Findings of the Association for Computational Linguistics: EMNLP 2023* (2023).

[7] Sun, Yutao, et al. "Retentive Network: A Successor to Transformer for Large Language Models." *arXiv preprint arXiv:2307.08621* (2023).

[8] Shazeer, Noam. "GLU Variants Improve Transformer." *arXiv preprint arXiv:2002.05202* (2020).

[9] Neftci, Emre O., Hesham Mostafa, and Friedemann Zenke. "Surrogate gradient learning in spiking neural networks." *IEEE Signal Processing Magazine* 36.6 (2019): 51-63.

[10] Kanerva, Pentti. *Sparse Distributed Memory*. MIT Press, 1988.

[11] Lample, Guillaume, et al. "Large memory layers with product keys." *Advances in Neural Information Processing Systems* 32 (2019).